

[Sign in](#)[VasaviDappili](#) / [Python_Problems](#) Public[Code](#)[Issues](#)[Pull requests](#)[Actions](#)[Projects](#)[Files](#)[main](#)[Python_Problems](#) / [README.md](#)[VasaviDappili](#) Update README.md

dd220a0 · 1 minute ago



271 lines (192 loc) · 8.2 KB

[Preview](#)[Code](#)[Blame](#)[Raw](#)

Python_Problems

1. Student Attendance Analysis

A college maintains the daily attendance details of its students in the form of a list containing student IDs. Some students may have attended multiple sessions on the same day. The administration wants to identify the longest continuous sequence of sessions in which no student ID is repeated. Develop a solution that determines the maximum length of such a sequence.

Code

```
students = [101, 102, 103, 101, 104, 105]

seen = set()

left = 0
longest = 0
```



```
for right in range(len(students)):

    while students[right] in seen:
        seen.remove(students[left])
        left += 1

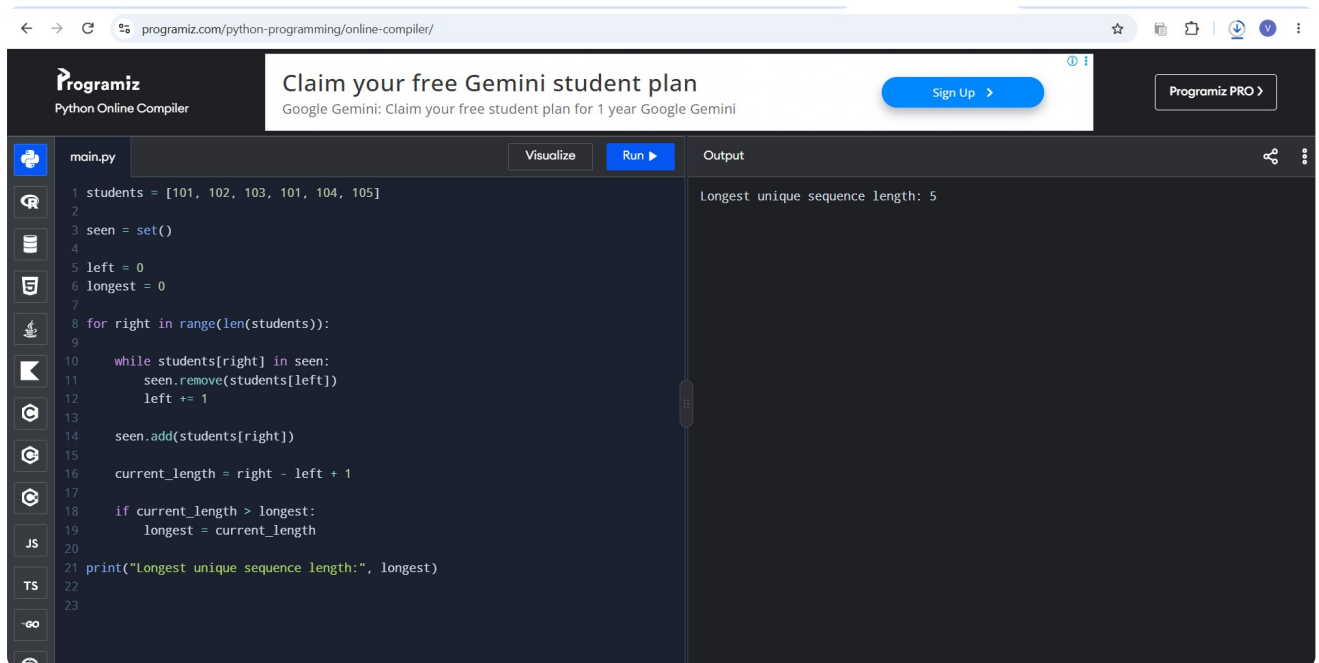
    seen.add(students[right])

    current_length = right - left + 1

    if current_length > longest:
        longest = current_length

print("Longest unique sequence length:", longest)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 students = [101, 102, 103, 101, 104, 105]
2
3 seen = set()
4
5 left = 0
6 longest = 0
7
8 for right in range(len(students)):
9
10     while students[right] in seen:
11         seen.remove(students[left])
12         left += 1
13
14     seen.add(students[right])
15
16     current_length = right - left + 1
17
18     if current_length > longest:
19         longest = current_length
20
21 print("Longest unique sequence length:", longest)
22
23
```

The output panel on the right displays the result of the execution:

```
Longest unique sequence length: 5
```

2. Online Shopping Price Analysis

An online shopping application stores the prices of products viewed by a customer during a browsing session. The customer wants to identify a continuous range of products that provides the maximum possible total discount value. Given the discount values, determine the maximum value that can be obtained from any continuous range.

Code

```
numbers = [2, -1, 3, 4, -2]

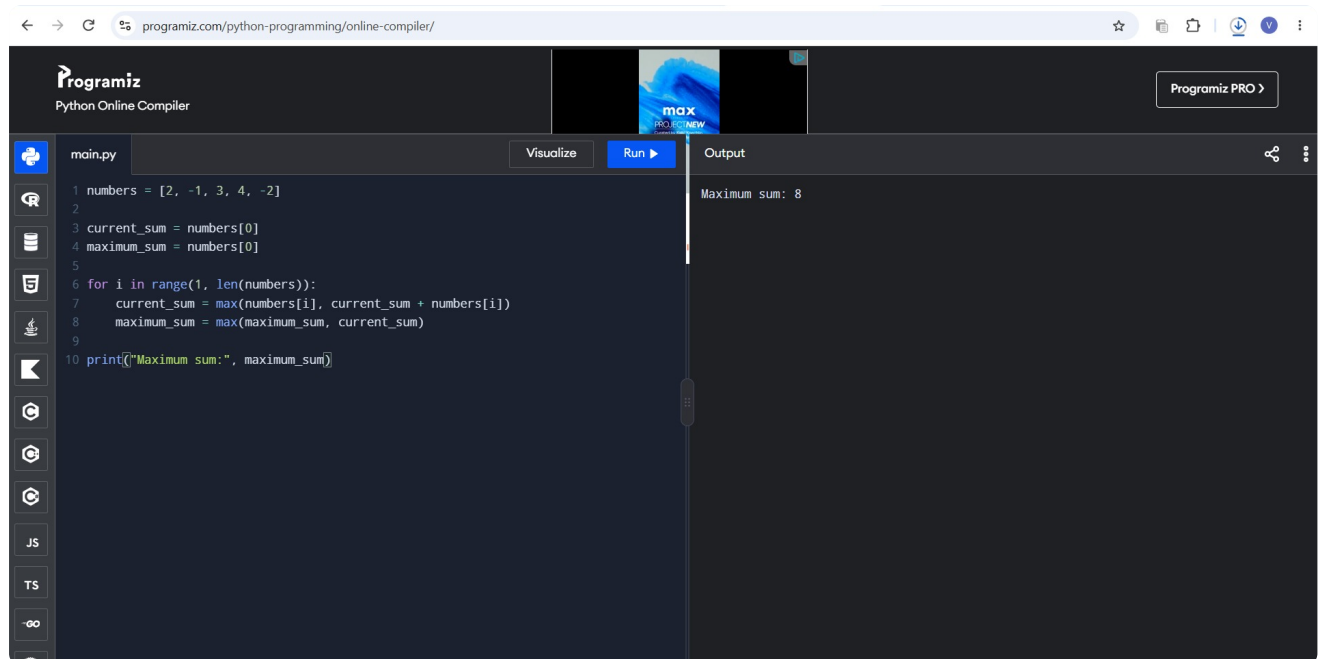
current_sum = numbers[0]
maximum_sum = numbers[0]

for i in range(1, len(numbers)):
    current_sum = max(numbers[i], current_sum + numbers[i])
    maximum_sum = max(maximum_sum, current_sum)

print("Maximum sum:", maximum_sum)
```



Output



3. Rainwater Collection System

A city installs buildings of different heights along a straight road. During rainfall, water gets collected between taller buildings. The engineering team needs to calculate the total amount of water that can remain trapped after heavy rainfall based on the heights of the buildings.

Code

```
heights = [3, 0, 2, 0, 4]

left = 0
right = len(heights) - 1

left_max = 0
right_max = 0

water = 0

while left < right:

    if heights[left] <= heights[right]:

        if heights[left] >= left_max:
            left_max = heights[left]
        else:
            water += left_max - heights[left]

        left += 1

    else:

        if heights[right] >= right_max:
            right_max = heights[right]
        else:
            water += right_max - heights[right]

        right -= 1

print("Trapped water:", water)
```

Output

The screenshot shows the Programiz Python Online Compiler interface. The code from the previous block is pasted into the editor. The 'Run' button is clicked, and the output is displayed in the right-hand pane. The output is 'Trapped water: 7'. The interface also includes a GoDaddy advertisement and a 'Programiz PRO' button.

```
Trapped water: 7
```

```
23 else:
24     if heights[right] >= right_max:
25         right_max = heights[right]
26     else:
27         water += right_max - heights[right]
28
29     right -= 1
30
31 print("Trapped water:", water)
```

4. Employee Performance Analysis

A company stores the monthly performance scores of an employee for several months. The scores may contain both positive and negative values depending on the employee's performance. Management wants to identify the continuous period during which the employee achieved the highest overall performance.

Code

```
scores = [-2, 3, -1, 5, -4]

best = scores[0]
total = scores[0]

for score in scores[1:]:
    if total < 0:
        total = score
    else:
        total = total + score

    if total > best:
        best = total

print("Highest performance score:", best)
```

Output

The screenshot shows a web browser at the URL `programiz.com/python-programming/online-compiler/`. The page features the Programiz logo and a banner for Godaddy. The main content area displays a Python code editor with the following code:

```
main.py
1 scores = [-2, 3, -1, 5, -4]
2
3 best = scores[0]
4 total = scores[0]
5
6 for score in scores[1:]:
7     if total < 0:
```

Below the code editor, the output is displayed:

```
Output
Highest performance score: 7
```

```
8     total = score
9     else:
10        total = total + score
11
12    if total > best:
13        best = total
14
15    print("Highest performance score:", best)
```

5. Product Sales Analysis

A retail company stores the daily sales quantity of a product for several consecutive days. Due to seasonal changes, some days may have negative adjustments. The company wants to identify the period that produced the highest multiplication of sales-related values. Develop a solution to determine this maximum product.

Code

```
values = [2, 3, -2, 4]

maximum = values[0]
minimum = values[0]
answer = values[0]

for value in values[1:]:

    old_max = maximum
    old_min = minimum

    maximum = max(value, value * old_max, value * old_min)
    minimum = min(value, value * old_max, value * old_min)

    answer = max(answer, maximum)

print("Maximum product:", answer)
```

Output

Programiz
Python Online Compiler

Get email that looks
as professional as
your business is.



Programiz PRO >

main.pyVisualizeRun

```
1 values = [2, 3, -2, 4]
2
3 maximum = values[0]
4 minimum = values[0]
5 answer = values[0]
6
7 for value in values[1:]:
8
9     old_max = maximum
10    old_min = minimum
11
12    maximum = max(value, value * old_max, value * old_min)
13    minimum = min(value, value * old_max, value * old_min)
14
15    answer = max(answer, maximum)
16
17 print("Maximum product:", answer)
```

Output
Maximum product: 6

6. Customer Purchase History

An e-commerce application stores the product IDs purchased by a customer in chronological order. The same product may appear multiple times. The system needs to determine the longest sequence of consecutive purchases in which every product ID is unique.

Code

```
products = [10, 20, 30, 20, 40, 50]

seen = set()
start = 0
longest = 0

for end in range(len(products)):

    while products[end] in seen:
        seen.remove(products[start])
        start += 1

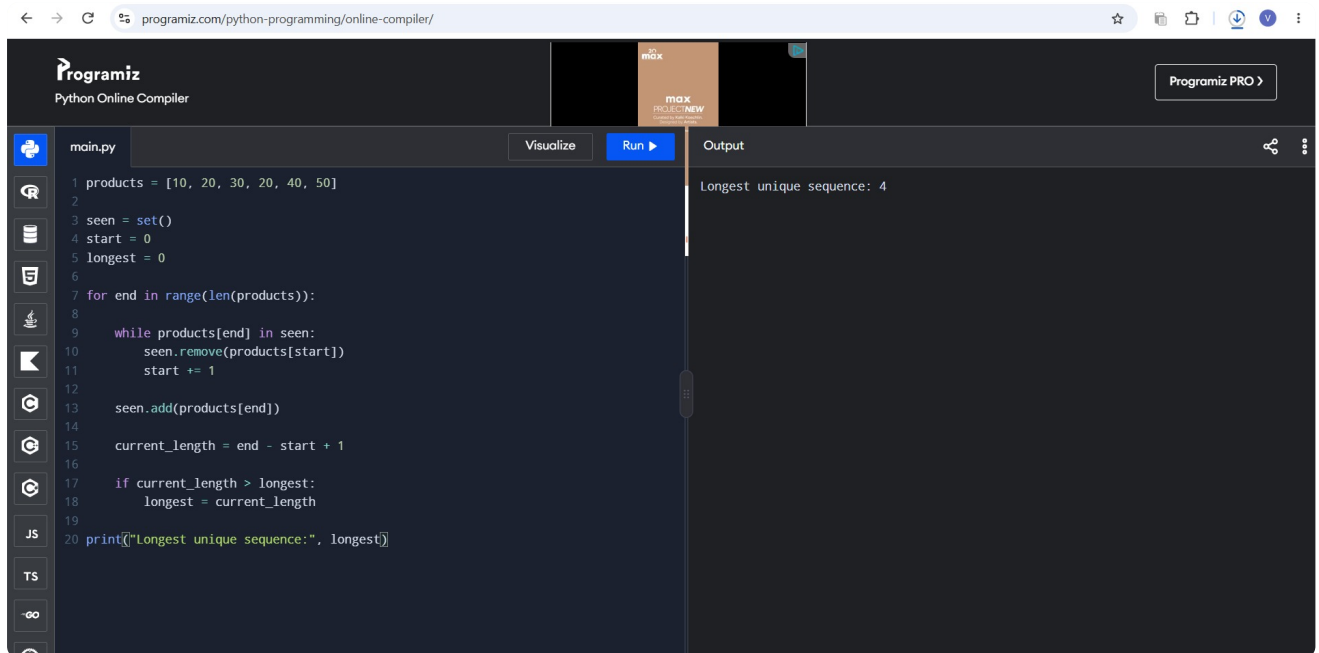
    seen.add(products[end])

    current_length = end - start + 1

    if current_length > longest:
        longest = current_length
```

```
print("Longest unique sequence:", longest)
```

Output



```
1 products = [10, 20, 30, 20, 40, 50]
2
3 seen = set()
4 start = 0
5 longest = 0
6
7 for end in range(len(products)):
8     while products[end] in seen:
9         seen.remove(products[start])
10        start += 1
11    seen.add(products[end])
12
13    current_length = end - start + 1
14
15    if current_length > longest:
16        longest = current_length
17
18 print("Longest unique sequence:", longest)
```

Output

Longest unique sequence: 4

7. Bank Transaction Analysis

A bank stores transaction amounts for a customer's account. A continuous group of transactions may add up to a specific target amount. The auditing system needs to determine how many different continuous transaction groups produce exactly the specified amount.

Code

```
transactions = [1, 2, 3, 2]
target = 5

prefix_sum = 0
count = 0

frequency = {0: 1}

for amount in transactions:
    prefix_sum += amount
```

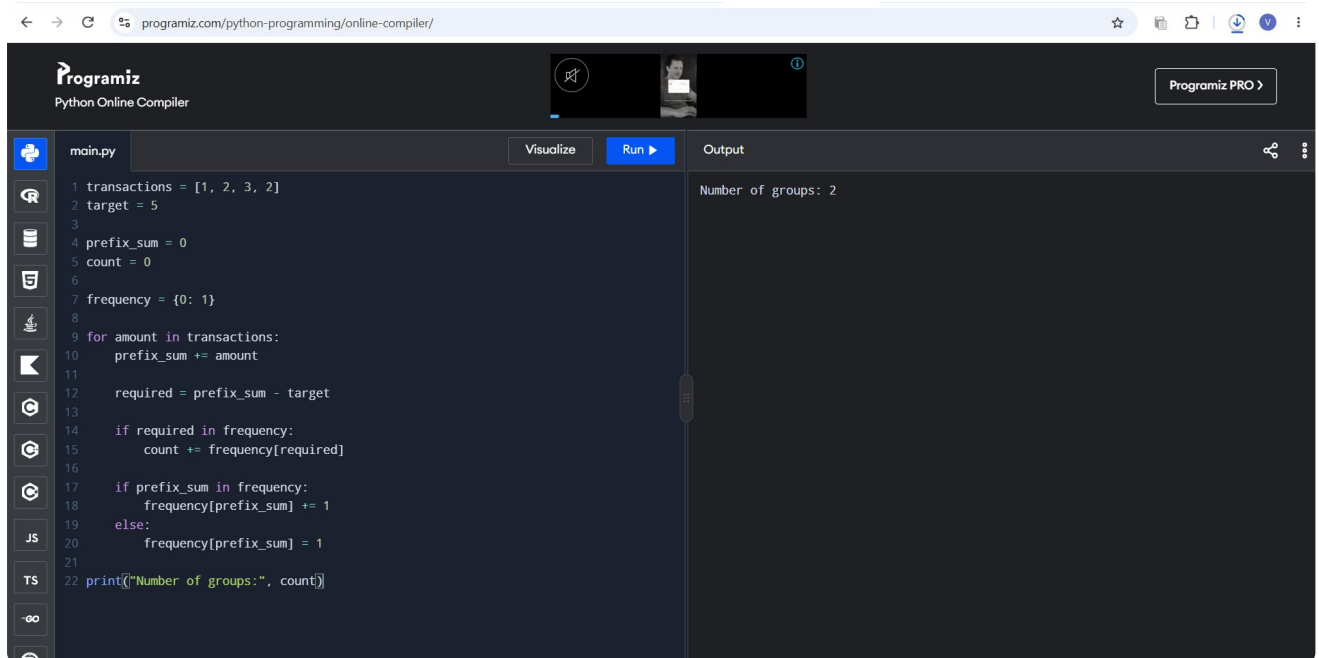
```
required = prefix_sum - target

if required in frequency:
    count += frequency[required]

if prefix_sum in frequency:
    frequency[prefix_sum] += 1
else:
    frequency[prefix_sum] = 1

print("Number of groups:", count)
```

Output



8. Employee Skill Grouping

A company receives a list of employee skill codes represented as strings. Employees having the same set of characters in their skill codes belong to the same skill category, even if the characters appear in a different order. The HR system needs to organize employees into appropriate skill groups.

Code

```
words = ["eat", "tea", "ate", "dog", "god", "cat"]

groups = {}

for word in words:
    key = "".join(sorted(word))

    if key not in groups:
        groups[key] = []

    groups[key].append(word)

for group in groups.values():
    print(group)
```



Output

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 words = ["eat", "tea", "ate", "dog", "god", "cat"]
2
3 groups = {}
4
5 for word in words:
6     key = "".join(sorted(word))
7
8     if key not in groups:
9         groups[key] = []
10
11     groups[key].append(word)
12
13 for group in groups.values():
14     print(group)
```

The right side of the interface shows the output of the code execution:

```
['eat', 'tea', 'ate']
['dog', 'god']
['cat']
```

9. Network Packet Analysis

A network monitoring system receives packet identifiers in chronological order. The system must determine the longest sequence of consecutive packets whose identifiers form a continuous numerical sequence, regardless of their original order in the incoming data.

Code

```
numbers = [100, 4, 200, 1, 3, 2]

number_set = set(numbers)
longest = 0

for number in number_set:

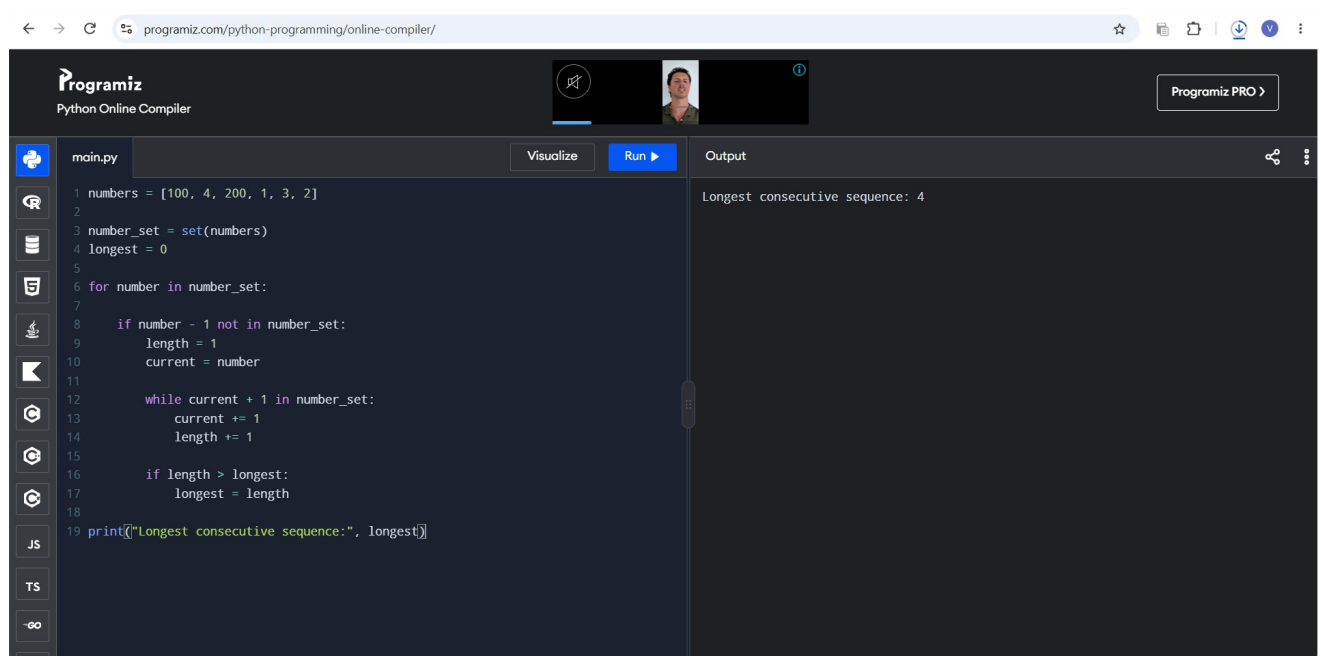
    if number - 1 not in number_set:
        length = 1
        current = number

        while current + 1 in number_set:
            current += 1
            length += 1

        if length > longest:
            longest = length

print("Longest consecutive sequence:", longest)
```

Output



10. Hospital Appointment Scheduling

A hospital receives appointment requests represented by

starting and ending times. Some appointments overlap with each other. The scheduling system needs to combine overlapping appointment periods so that the final schedule contains only non-overlapping time ranges.

Code

```
appointments = [[1, 3], [2, 5], [7, 9]]

appointments.sort()

merged = []

for appointment in appointments:

    if not merged or appointment[0] > merged[-1][1]:
        merged.append(appointment)
    else:
        merged[-1][1] = max(merged[-1][1], appointment[1])

print("Merged appointments:", merged)
```

Output

